BoxLite PR Summary

Submitted PRs

PR #406 — fix(jailer): Dynamic FD Closure

Item	Detail
URL	https://github.com/boxlite-ai/boxlite/pull/406
Branch	`fix/jailer-dynamic-fd-closure`
Commit	`28e2ce4`
Category	Security fix
Files changed	1 file, +257 / -15

Problem: FD cleanup in jailer `pre_exec` used hardcoded upper bounds (1024 on Linux, 4096 on macOS). On systems with raised `ulimit -n`, FDs above these limits leaked into jailed processes, potentially exposing credentials, database connections, or network sockets.

Solution: 3-strategy cascade (Linux):

1. `close_range(first_fd, ~0U, 0)` — O(1), Linux 5.9+
2. `/proc/self/fd` enumeration via raw `getdents64` — no heap allocation
3. Brute-force close with dynamic limit from `getrlimit(RLIMIT_NOFILE)`

macOS uses brute-force with dynamic `getrlimit` limit. All operations remain async-signal-safe for the `pre_exec` context.

PR #407 — feat(vmm): pidfd/kqueue Event-Driven Process Monitor

Item	Detail
URL	https://github.com/boxlite-ai/boxlite/pull/407
Branch	`feat/pidfd-kqueue-process-monitor`
Commit	`78d484e`
Category	Performance
Files changed	2 files, +467 / -18

Problem: `ProcessMonitor::wait_for_exit()` used a 500ms sleep-based polling loop (`tokio::time::sleep` + `try_wait`), violating Rule #15: "No Sleep for Events." This added up to 500ms latency to VM crash detection during startup.

Solution: Platform-native event-driven mechanisms:

- **Linux**: `pidfd_open()` (kernel 5.3+) + tokio `AsyncFd`
- **macOS**: `kqueue` + `EVFILT_PROC` + `NOTE_EXIT` + tokio `AsyncFd`
- **Fallback**: 100ms polling for older kernels (< 5.3)

Key design: `OwnedFd` wraps raw FDs immediately (leak-free by construction), `fcntl O_NONBLOCK` with graceful fallback, best-effort race guard via `is_alive()` after FD setup.

PR #408 — feat(python-sdk): EventListener + Typed Errors

Item	Detail
URL	https://github.com/boxlite-ai/boxlite/pull/408
Branch	`feat/python-event-listener-typed-errors`
Commit	`e5ad727`
Category	SDK feature
Files changed	17 files, +1050 / -75

Problem: Python SDK had no way to receive push-based lifecycle callbacks. All errors were generic `PyRuntimeError`, making programmatic error handling impossible.

Solution:

- **PyEventListener** bridge: duck-typing via PyO3, missing methods silently skipped
- **Typed exceptions**: 15 exception classes inheriting from `BoxliteError`, exhaustive match on all 18 `BoxliteError` variants for compile-time completeness
- `event_listeners` parameter on `BoxliteOptions`, propagated through `RuntimeImpl`
- **165 Python tests** covering exception hierarchy, isolation, and exports

PR #409 — feat(portal): Streaming File Upload

Item	Detail
URL	https://github.com/boxlite-ai/boxlite/pull/409
Branch	`feat/streaming-file-upload`
Commit	`37ca16f`
Category	Performance
Files changed	1 file, +365 / -36

Problem: `upload_tar` buffered the entire file into a `Vec`, causing memory usage of O(file_size). Large file uploads could OOM the host process.

Solution: Bounded `mpsc` channel (capacity=4) with a spawned reader task, capping peak memory at ~5 MiB regardless of file size. Matches the streaming pattern already used in `download_tar` and guest-side upload handler.

Key design: `stream_file_chunks` helper accepts `impl AsyncRead` for testability, `std::mem::take` for zero-copy first chunk, always await reader `JoinHandle` before checking gRPC result (root-cause priority). 8 unit tests added.

Summary Stats

PR	Category	Lines Changed
#406 Dynamic FD Closure	Security	+257 / -15
#407 pidfd/kqueue ProcessMonitor	Performance	+467 / -18
#408 Python EventListener + Typed Errors	SDK	+1050 / -75
#409 Streaming File Upload	Performance	+365 / -36
Total		**+2139 / -144**